Sukijan asennus- ja käyttöohje

Sukija on Javalla kirjoitettu Solr:n liitännäinen suomenkielisten tekstien indeksointiin.

Sukija analysoi sanat morfologisesti, muuttaa sanat perusmuotoon (joka on sanakirjoissa) ja indeksoi perusmuodot, jotta sanan kaikki taivutusmuodot löytyvät vain perusmuotoa etsimällä.

Sukija tallentaa perusmuodot Solr:n tietokantaan, josta niitä voi etsiä Solr:n käyttöliittymän kautta.

Sukija osaa indeksoida kaikkia niitä tiedostomuotoja ja tekstitiedostojen merkistökoodauksia, joita Apache Tika (`http://tika.apache.org/`) osaa lukea.

Mitä tarvitaan ja mistä ne saa?

- Sukija: `https://github.com/ahomansikka/sukija` Koska luet tätä tekstiä, olet jo imuroinut tämän. (-:
- Corevoikko: `https://github.com/voikko/corevoikko` Uusin versio. C-koodin pitää olla käännetty niin, että sanojen perusmuodot tulevat mukaan.
- Solr 10.1.0: `http://lucene.apache.org/solr/` Tässä dokumentissa Solr:sta käytetään nimeä solr-x.y.z, missä x.y.z tarkoittaa version numeroa.
- Apache Tika Server: `https://tika.apache.org/` Sukija ei tarvitse tätä, mutta Solr tarvitsee. Mieluiten kai uusin versio. Tämä pitää asentaa ennen Solr:ia.
- Javan versio 25 tai uudempi. Sukija ei ehkä käänny Javan aiemmilla versiolla.
- Apache maven.
- Apache Lucene ja Apache Tika. Ne tulevat Solr:n mukana, mutta Sukijan tiedostosta `pom.xml` on hyvä tarkistaa, että paketeista `lucene-core`, `lucene-suggest` ja `tika-core` on siellä samat versiot, joita solr-x.y.z käyttää.

Lisäksi Sukija tarvitsee erinäisiä jar-tiedostoja, mutta maven imuroi ne automaagisesti.

Sorl:n asentaminen on hieman muuttunut edellisestä versiosta ja siis myös nämä asennusohjeet.

Tämä asennusohje olettaa, että libvoikko on asennettu hakemistoon `/usr/local/lib` ja että Solr on hakemistossa `$HOME/Lataukset/solr/solr-x.y.z` Siellä Solr:n käyttämät Lucenen ja Tikan versiot saa esille Linuxissa näin

```
find . -regextype egrep -regex '.*(lucene|tika).*jar'
```

Apache Tika Serverin asentaminen

`https://cwiki.apache.org/confluence/display/TIKA/TikaServer`

Solr:n asentaminen

Solr:ia ei tarvitse asentaa, vaan sitä voi käyttää suoraan hakemistosta `$HOME/Lataukset/solr/solr-x.y.z`.

Tuotantokäyttöä varten Solr on parasta asentaa palveluna (service). Tämä ohje olettaa, että Solr on asennettu palveluna.

Katso

```
https://solr.apache.org/guide/solr/latest/
Deployment Guide -> Installation & Deployment
 -> Taking Solr to Production
```

Jos kone on kiinni verkossa, kannattaa katsoa myös ohjeet turvallisuudesta

```
https://solr.apache.org/guide/solr/latest/
Deployment Guide -> Securing Solr
```

Solr:n uusimmassa versiossa pyörii oletusarvoisesti monia palvelimia (SolrCloud Mode). Se on kuitenkin aivan liian raskas yhden koneen kovalevy(je)n indeksointiin, mutta ei hätää: se voidaan konfiguroida vain yhden palvelimen pyörittämiseen.

Jotta Solr käynnistyisi automaagisesti vain yhdellä palvelimella, tiedostossa `/etc/systemd/system/solr.service` rivi

```
ExecStart=/opt/solr/bin/solr start
```

pitää muuttaa muotoon

```
ExecStart=/opt/solr/bin/solr start --user-managed
```

ja tehdä sen jälkeen

```
sudo systemctl daemon-reload
```

Käsipelillä komennot ovat

```
/opt/solr/bin/solr start --user-managed
/opt/solr/bin/solr restart --user-managed
```

Sukijan asentamiseksi tiedostossa `/etc/default/solr.in.sh` rivi

`#SOLR_MODULES=extraction,ltr`

pitää muuttaa muotoon

`SOLR_MODULES=extraction,ltr,scripting`

Lisäksi tiedoston

`/opt/solr/server/etc/security.policy`

loppuun pitää lisätä tiedosto

`src/main/resources/security.policy`

Sukijan kääntäminen ja asentaminen

Sukija käännetään ja asennetaan komennolla

```
mvn install
```

Komento imuroi netistä tarvitsemansa Javan jar-paketit eli ensimmäinen kääntäminen saattaa kestää kauan. Erityisen kauan se kestää, jos et ole aiemmin käyttänyt mavenia.

Sukijan jar-tiedosto asennetaan maven-hakemistoon `${HOME}/.m2`

Sukijan konfigurointi

Sukijan parametrit asetetaan tiedostossa `sukija.properties`, jossa on myös tarvittavat ohjeet.

Sen jälkeen komennolla

```
make tiedostot
```

tehdään Solr:n asetustiedosto `conf/schema.xml`. Esimerkin siitä saa aikaan yllä mainitulla komennolla. (-:

Seuraavaksi pitää tehdä Solr:iin Sukijan tarvitsemat hakemistot ja kopioida niihin Solr:n tarvitsemat tiedostot. Sitä varten on kaksi skriptiä

```
src/main/resources/install.sh
src/main/resources/install-sudo.sh
```

Jälkimmäistä käytetään, jos Solr on asennettu tunnuksella `solr`.

Skriptit asentavat Sukijan tiedostot kansioon `/var/solr/data/configsets`

Asennuksen jälkeen tiedostoja voi muokata joko hakemistossa `/var/solr/data/configsets/conf` tai Sukijan lähdekoodissa ja kopioida tuohon kansioon, ja päivitetyt jar-paketit voi kopioida kansioon `/var/solr/data/configsets/lib`

Päivityksen jälkeen Solr pitää käynnistää uudellen. Linuxissa se käy näin

```
/opt/solr/bin/solr start --user-managed
```

Jos Solr:ia käyttää selaimen kautta, sekin on syytä käynnistää uudelleen.

Seuraavaksi mennään selaimella osoitteeseen
`http://localhost:8983/solr/`

Vasemmalla paneelin alalaidassa lukee

```
No cores available
Go and create one
```

Mennään sinne ja kirjoitetaan kohtaan `name` ja `instanceDir` ”sukija” (mutta ilman lainausmerkkejä) ja napsautetaan kohtaa `Add Core`.

Nyt vasemmalle paneeliin alalaitaan pitäisi tulla mahdollisuus valita indeksi (Solr käyttää siitä nimeä `core`) sukija. Sen jälkeen voi ruveta indeksoimaan. Katso sivu 7.

Sukijan loki

Sukijan lokitulostus konfiguroidaan Solr:n lokitulostuksen kautta tiedostossa `/var/solr/log4j2.xml`. Sukijan käyttämät lisäykset ovat on hakemistossa `src/main/resources/log4j2.xml`. Ne on kommentoitu pois, mutta niitä voi siirtää halutun määrän kommenttien ulkopuolelle. Jotta Solr näkisi muutokset, ne pitää laittaa tiedostoon `/var/solr/log4j2.xml`.

Lokitulostuksen eri tasot (`all`, jne.) voi katsoa Solr:n dokumenteista.

Lokitulostus menee kansioon `/var/solr/logs/`

Indeksointi

Vihje: Myös `Apache Nutch`in voi konfiguroida indeksoimaan kovalevyn eli paikallisen tiedostojärjestelmän.

Versiosta 9.0.0 alkaen Solr:sta on poistettu `DataImportHandler`, jota käytin kovalevyni indeksointiin, ja sen korvaajaksi tein ohjelman

```
src/main/resources/disk-indexer.sh <asetustiedosto>
```

Se toimii minulla, mutta en väitä, että se toimii sinulla.

Se valitsee tiedostot niitten nimien perusteella. Valinta MIME"-tyyppien pohjalta täytyy tehdä Apache Tika Serverin asetuksissa.

Asetustiedoston esimerkkinä on

```
src/main/resources/indexer-config-esim.xml
```

Tiedostoa voidaan muokata XML-editorilla tai emacsin XML-moodilla. Tiedoston kentät ovat

solrURL	Solr:n URL.
collection	Tietokannan nimi (sukija).
commitWithinMs	Enimmäisaika millisekunteina ennen kuin indeksoitavat tiedostot tallenetaan Solr:ään.
connectionTimeout	Nämä kolme kenttää (millisekunteina) liittyvät
idleTimeout	HTTP-yhteyksiin. Katso luokan
requestTimeout	`HttpSolrClientBuilderBase` dokumentti.
onError	Mitä tehdään virheilmoituksen jälkeen? `abort` lopettaan. `skip` siirrytään indeksoimaan seuraavaa tiedostoa.
baseDir	Indeksoitavat hakemistot, jokainen erikseen, merkkijonoina.
includes	Säännölliset lausekkeet, joilla valitaan indeksoitavat tiedostot (ei välttämätön).
excludes	Säännölliset lausekkeet, joilla valitaan tiedostot, joita ei indeksoida (ei välttämätön).

Jos `includes` ja `excludes` puuttuvat, indeksoidaan kaikki hakemistoissa `baseDir` olevat tiedostot.

Esimerkkejä

```
<baseDir>${user.home}/Asiakirjat</baseDir>
```

```
<pattern>.*(12345|abcd).*</pattern>
```

Valtiaan tiedostot, joitten nimissä on `12345` tai `abcd`

```
<pattern>${user.home}/Asiakirjat/salainen/.*</pattern>
```

Valitaan kaikki hakemiston tiedostot.

Syntaksi on sama kuin Javan luokassa `Pattern` ja `user.home` on ainoa tunnistettu Javan muuttuja. Säännöllisen lausekkeen pitää testata koko tiedoston nimeä eli `.*` on tarpeen alussa tai lopussa (tai `$`).

Jos tiedostot valitaan niitten loppuosan perusteella, säännöllinen lauseke on jotain `.*[.](odt|tex|txt)$` Tällaisen lausekkeen kirjoittamisessa tulee helposti virheitä varsinkin, jos loppuosia on paljon (`latexmk` tekee paljon aputiedostoja). Niinpä jokainen tiedoston loppuosa voidaan laittaa omalle rivilleen merkkijonona.

```
<extension>odt</extension>
```

ja ohjelma kääntää sen toiseksi säännölliseksi lausekkeeksi `pattern`in (jos sellainen on) lisäksi.

`includes`-osassa valitaan kaikki ne tiedostot, jotka jompikumpi säännöllinen lauseke valitsee.

`excludes`-osassa valitaan kaikki ne tiedostot, joita kumpikaan säännöllinen lauseke ei valitse.

Sukija ei indeksoi tar-, zip- tai muitten sellaisten tiedostojen sisällä olevia tiedostoja.

Tietojen etsiminen

Sanoja etsitään menemällä osoitteeseen
`http://localhost:8983/solr/sukija/browse`

Etsittävien sanojen tulee olla perusmuodossa (yksikön nominatiivissa) ja pienellä kirjoitettuna (myös erisnimet). Etsittäessä sanoja ei muuteta perusmuotoon siksi, että yhden sanan perusmuoto voi olla toisen sanan taivutusmuoto. Paras esimerkki tästä on ”alusta”, joka on sanojen ”alusta”, ”alustaa”, ”alku”, ”alunen” ja ”alus” taivutusmuoto. Tällöin herää kysymys, mitä sanaa pitää etsiä, vai etsitäänkö kaikkia?

Myös monikolliset sanat on tallennettu yksikössä. Esimerkiksi Yhdysvaltojen perusmuoto on Yhdysvalta.

Eri tavalla muotoillun tulostuksen saa osoitteesta
`http://localhost:8983/solr/sukija/select`

Esimerkiksi sanaa `sana` etsitään näin:
`http://localhost:8983/solr/sukija/select?q=sana`

Tämän tulostuksen ulkonäköä voi muuttaa muuttamalla Sukijan mukana tulevaa tiedostoa `conf/xslt/sukija.xsl`.

Tietoja voi etsiä myös Solr:n käyttöliittymällä kohdassa `Query`.

Koska sanat on tallennettu perusmuodossa, fraasihaussakin sanojen täytyy olla perusmuodossa. Esimerkiksi Helsingin yliopistoa pitää etsiä fraasihaulla

`"helsinki yliopisto"`

Indeksoinnissa w:t muutetaan v:ksi eli esimerkiksi Waltaria on etsittävä muodossa

`valtari`

Tiedoston suggestions.xml konfigurointi

Tässä vaiheessa kaikki indeksoitavista tiedostoista luetut sanat on muutettu pieniksi kirjaimiksi eli tiedostossa `suggestions.xml` olevat erisnimetkin pitää kirjoittaa pienellä alkukirjaimella.

Konfigurointitiedoston formaatti on määritelty tiedostossa `/src/main/xsd/SuggestionInput.xsd`.

Konfigurointitiedostossa olevien säännöllisten lausekkeiden syntaksi on sama kuin Javan luokassa `java.util.regex.Pattern`

Konfiguroinnille ei ole käyttöliittymää, koska tiedoston `suggestions.xml` muokkaamiseen voi käyttää mitä tahansa XML-editoria.

`compoundWordEnd` tunnistaa yhdyssanan, jos se loppuu tiettyyn sanaan. Esimerkiksi

```
<compoundWordEnd>
  <input>joki joki</input>
  <input>joke joki</input>
  <input>joe joki</input>
</compoundWordEnd>
```

tunnistaa esimerkiksi merkkijonon ”aatsajoelle”. Tällä tavalla voidaan tunnistaa paikannimiä, jotka eivät ole sanastossa.

Jokaisessa input-lauseessa on kaksi osaa. Ensimmäinen on jokin merkkijono ja toinen jonkin sanan perusmuoto. Tunnistettaessa merkkijono katkaistaan siitä kohdasta, josta ensimmäinen merkkijono alkaa, ja jos merkkijonon loppuosan perusmuoto on argumentin toinen osa, perusmuotona palautetaan merkkijonon alkuosa + argumentin toinen osa.

Esimeriksi ”aatsajoelle” jaetaan kahtia osiin ”aatsa” ja ”joelle”, ja koska merkkijonon ”joelle” perusmuoto on ”joki”, merkkijonon ”aatsajoelle” perusmuodoksi tulee ”aatsajoki”.

Input-lauseita voi olla mielivaltainen määrä.

Ensimmäinen merkkijono ei ole säännöllinen lauseke, koska ne ovat hitaampia kuin merkkijonot varsinkin, kun merkkijonojen tunnistuksessa käytetään Aho-Corasick -algoritmia.[1] Algoritmissa sanan (ensimmäisen merkkijonon) etsimiseen käytetty aika ei riipu etsittävien sanojen määrästä, sillä se etsii kaikkia sanoja samanaikaisesti.

`prefix` tunnistaa etuliitteettömän sanan (”etuliite” voi olla mikä tahansa merkkijono). Esimerkiksi

```
<prefix>
  <prefix>abcdefg</prefix>
  <savePrefix>true</savePrefix>
  <saveWord>true</saveWord>
</prefix>
```

[1] Katso Alfred V. Aho and Margaret J. Corasick: *Efficient String Matching: An Aid to Bibliographic Search.* Communications of the ACM. June 1975 Volume 18 Number 6. Löytyy netistä googlaamalla.

poistaa sanan alusta merkkijonon ”abcdefg” ja yrittää tunnistaa jäljelle jääneen sanan (”abcdefgsuomalaiselle” => ”suomalaiselle”) ja tallentaa sen perusmuodon (”suomalainen”). Jos `savePrefix` on true, tallentaa myös etuliitteen (”abcdefg”) ja jos `saveWord` on true, tallentaa myös koko sanan perusmuodon (”’abcdefgsuomalainen”).

Prefix-lauseita voi olla mielivaltainen määrä.

`char` muuttaa sanassa olevat merkit toiseksi. Tämä vastaa Unixin komentoa `tr`. Esimerkiksi

```
<char>
  <from>gbdkptvw</from>
    <to>kptgbdwv</to>
</char>
```

muuttaa g:n k:ksi, b:n p:ksi jne. Ohjelma testaa muutettavien kirjainten kaikki mahdolliset kombinaatiot. Esimerkiksi jos tiedostosta luettu sana on ”piolokia”, komento yrittää tunnistaa sanat ”piolokia”, ”biolokia”, ”piologia” ja ”biologia” (mutta ei välttämättä tässä järjestyksessä :-).

`regex` muuttaa säännöllisen lausekkeen. Esimerkiksi

```
<regex>
 <input>(ai)(j)([eou])   $1$3</input>
 <input>^([0-9]+)</input>
 <tryAll>true</tryAll>
</regex>
```

Ensimmäinen input-lause poistaa j-kirjaimen muun muassa sanoista ”aijemmin”, ”aijomme” ja ”kaijutin”, ja toinen poistaa numerot sanan alusta.

Säännöllisessä lausekkeessa voi käyttää seuraavia lyhenteitä:

```
%A  [aä]
%C  [bcdfghjklmnpqrsštvwxzž]
%O  [oö]
%U  [uy]
%V  [aeiouyäö]
%%  %
```

Esimerkiksi `<input>(%V)(h)(%V) $1hd$3</input>` muuttaa h:n hd:ksi esimerkiksi sanassa ”puhistus”.

Jos `tryAll` on true, ohjelma kokeilee kaikkia säänöllisiä lausekkeita, jos se on false, ohjelma lopettaa ensimmäisen tunnistetun sanan jälkeen.

Input-lauseen ensimmäinen osa on säännöllinen lauseke ja toinen merkkijono, miksi se muutetaan. Sen syntaksi on sama kuin Javan luokassa `java.util.regex.Matcher`.

Jos toista osaa ei ole, säännöllisen lausekkeen tunnistama merkkijono poistetaan. Input-lauseita voi olla mielivaltainen määrä.

`start` käy läpi kaikki sanan alut pisimmästä (maxLength) alkaen lyhimpään (minLength) asti ja lopettaa, kun löytyy ensimmäinen tunnistettu sana

```
<start>
  <minLength>4</minLength>
  <maxLength>10</maxLength>
  <baseFormOnly>true</baseFormOnly>
</start>
```

Jos `baseFormOnly` on true palautetaan sana vain, jos se on perusmuodossa (esim. ”autowerwwwww” palauttaa ”auto”), muuten palautetaan tunnistettu sana muutettuna perusmuotoon (”kuudenwwwww” palauttaa ”kuusi”, mutta jos `baseFormOnly` on false, ei palauteta mitään).

`apostrophe` poistaa sanasta heittomerkin ja yrittää tunnistaa sanan sen jälkeen. Jos tunnistaminen ei onnistu, poistaa sanasta heittomerkin ja kaikki sen jälkeiset merkit ja palauttaa jäljelle jääneet merkit sanan perusmuotona. Esimerkiksi yrittää tunnistaa merkkijonon `centime'in` muodossa `centimein`. Jos tunnistaminen ei onnistu, palauttaa merkkijonon `centime`.

```
<apostrophe/>
```

Tällä komennolla ei ole parametreja.

Copyright © 2011–2026 Hannu Väisänen.